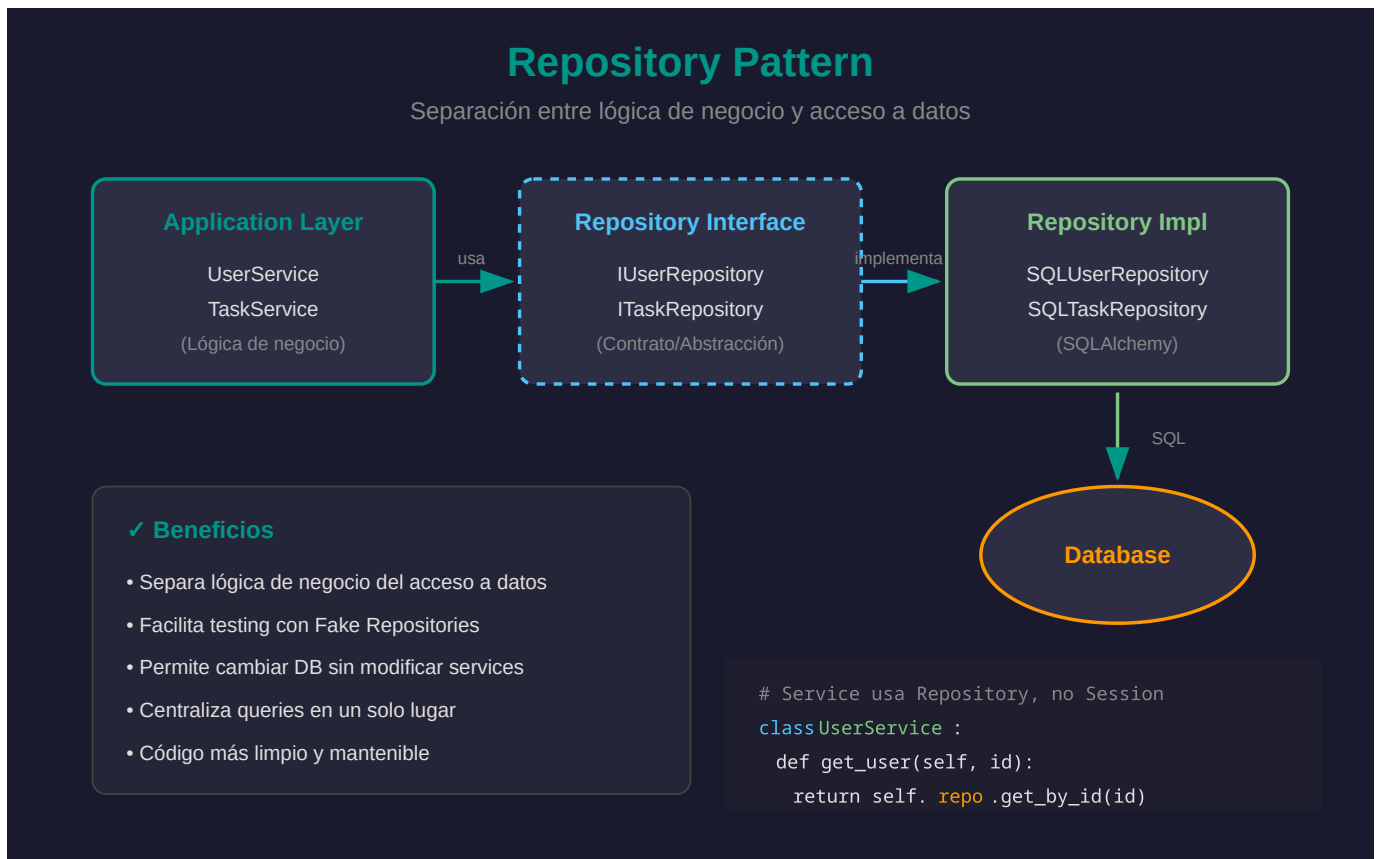


Introducción al Repository Pattern



Objetivos

- Entender qué es el Repository Pattern y su propósito
- Conocer los problemas que resuelve
- Ver la diferencia entre acceso directo a DB y usar repositorios
- Comprender cuándo aplicar este patrón

¿Qué es el Repository Pattern?

El **Repository Pattern** es un patrón de diseño que **abstrae el acceso a datos** detrás de una interfaz similar a una colección. El resto de la aplicación trabaja con objetos de dominio sin conocer los detalles de cómo se almacenan.

Definición de Martin Fowler

"Un repositorio media entre el dominio y las capas de mapeo de datos usando una interfaz similar a una colección para acceder a objetos del dominio."

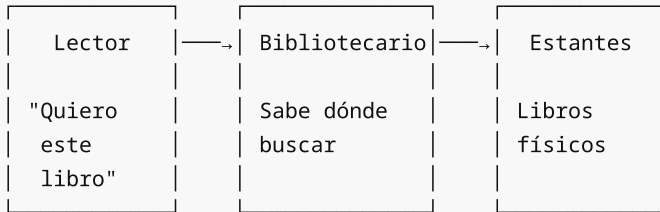
Analogía: El Bibliotecario

Imagina una biblioteca:

Sin repositorio (acceso directo):

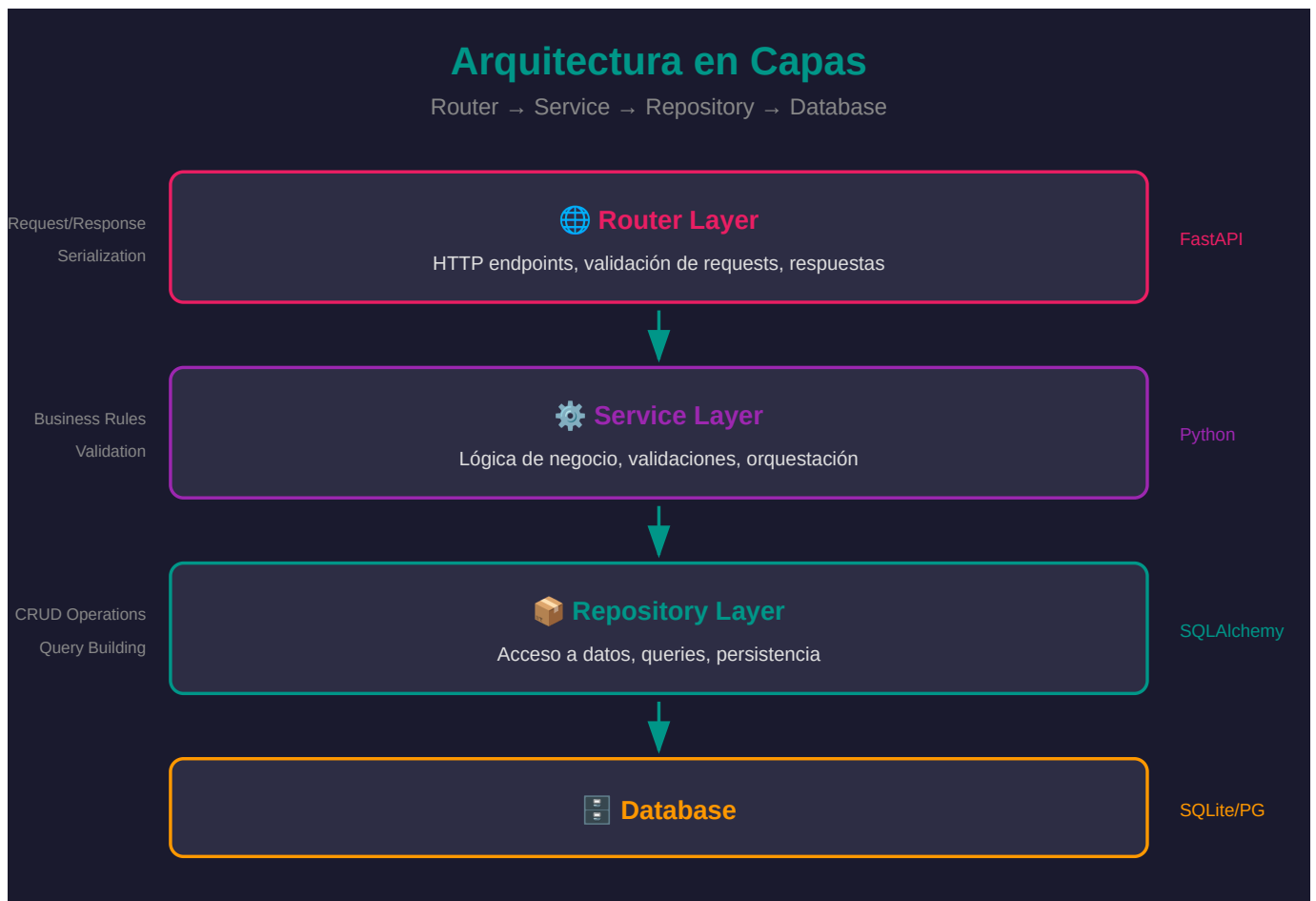
Lector	→	Buscar en estantes, catálogos
		Saber sistema de clasificación
		Conocer ubicación física

Con repositorio (bibliotecario):



El lector (service) solo pide el libro. El bibliotecario (repository) sabe cómo encontrarlo.

Arquitectura en Capas



🔍 Problema: Sin Repository Pattern

Código Típico de Week-06

En la semana anterior, nuestros servicios accedían directamente a SQLAlchemy:

```
# services/author_service.py - Week 06
from sqlalchemy import select
from sqlalchemy.orm import Session

from models import Author
```

```

class AuthorService:
    def __init__(self, db: Session):
        self.db = db

    def get_by_id(self, author_id: int) -> Author | None:
        # ❌ Service conoce detalles de SQLAlchemy
        return self.db.get(Author, author_id)

    def get_by_email(self, email: str) -> Author | None:
        # ❌ Service construye queries
        stmt = select(Author).where(Author.email == email)
        return self.db.execute(stmt).scalar_one_or_none()

    def list_all(self, skip: int = 0, limit: int = 10) -> list[Author]:
        # ❌ Lógica de paginación mezclada
        stmt = select(Author).offset(skip).limit(limit)
        return self.db.execute(stmt).scalars().all()

    def create(self, data: AuthorCreate) -> Author:
        # Validación de negocio ✅
        if self.get_by_email(data.email):
            raise DuplicateError("Email already exists")

        # ❌ Operaciones de persistencia en service
        author = Author(**data.model_dump())
        self.db.add(author)
        self.db.commit()
        self.db.refresh(author)
        return author

```

Problemas de este Enfoque

Problema	Descripción
Acoplamiento	Service está acoplado a SQLAlchemy
Testing difícil	Necesitas BD real para testear lógica de negocio
Duplicación	Mismas queries en múltiples services
Cambio costoso	Cambiar de ORM requiere modificar todos los services
Responsabilidades mezcladas	Lógica de negocio + acceso a datos

✅ Solución: Repository Pattern

Separación de Responsabilidades

```

# repositories/author_repository.py
from sqlalchemy import select
from sqlalchemy.orm import Session

from models import Author

```

```

class AuthorRepository:
    """Repositorio para operaciones de datos de Author"""

    def __init__(self, db: Session):
        self.db = db

    def get_by_id(self, author_id: int) -> Author | None:
        return self.db.get(Author, author_id)

    def get_by_email(self, email: str) -> Author | None:
        stmt = select(Author).where(Author.email == email)
        return self.db.execute(stmt).scalar_one_or_none()

    def list_all(self, skip: int = 0, limit: int = 10) -> list[Author]:
        stmt = select(Author).offset(skip).limit(limit)
        return self.db.execute(stmt).scalars().all()

    def add(self, author: Author) -> Author:
        self.db.add(author)
        self.db.flush() # Obtiene ID sin commit
        return author

    def delete(self, author: Author) -> None:
        self.db.delete(author)

```

```

# services/author_service.py - CON Repository
from models import Author
from schemas import AuthorCreate
from repositories import AuthorRepository
from exceptions import DuplicateError

class AuthorService:
    """Service para lógica de negocio de Author"""

    def __init__(self, author_repo: AuthorRepository):
        # ✅ Recibe repository, no Session
        self.repo = author_repo

    def get_by_id(self, author_id: int) -> Author | None:
        # ✅ Delega al repository
        return self.repo.get_by_id(author_id)

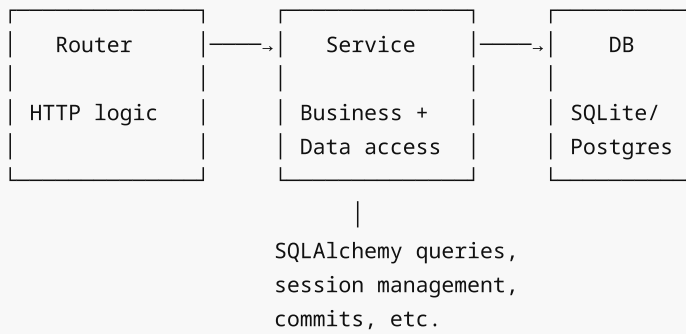
    def create(self, data: AuthorCreate) -> Author:
        # ✅ Solo lógica de negocio
        if self.repo.get_by_email(data.email):
            raise DuplicateError("Email already exists")

        # ✅ Crea objeto y delega persistencia
        author = Author(**data.model_dump())
        return self.repo.add(author)

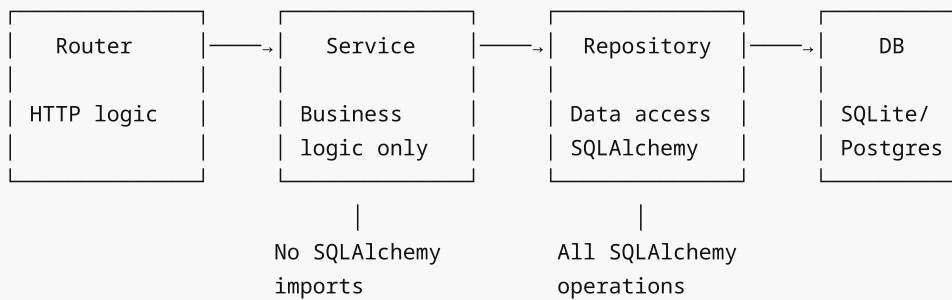
```



Antes (Week-06): Service → Database



Después (Week-07): Service → Repository → Database



🎯 Beneficios del Repository Pattern

1. Separación de Responsabilidades

```
# Service: SOLO lógica de negocio
class TaskService:
    def complete_task(self, task_id: int) -> Task:
        task = self.repo.get_by_id(task_id)
        if not task:
            raise NotFoundError("Task not found")

        # Regla de negocio
        if task.is_completed:
            raise BusinessError("Task already completed")

        task.is_completed = True
        task.completed_at = datetime.utcnow()
        return self.repo.update(task)

# Repository: SOLO acceso a datos
class TaskRepository:
    def get_by_id(self, task_id: int) -> Task | None:
        return self.db.get(Task, task_id)

    def update(self, task: Task) -> Task:
        self.db.flush()
        return task
```

2. Testing Simplificado

```
# Test SIN base de datos real
class FakeTaskRepository:
    def __init__(self):
        self.tasks = {1: Task(id=1, title="Test", is_completed=False)}

    def get_by_id(self, task_id: int) -> Task | None:
        return self.tasks.get(task_id)

    def update(self, task: Task) -> Task:
        self.tasks[task.id] = task
        return task

def test_complete_task():
    fake_repo = FakeTaskRepository()
    service = TaskService(task_repo=fake_repo)

    task = service.complete_task(1)

    assert task.is_completed is True
```

3. Reutilización de Queries

```
# Repository con métodos específicos reutilizables
class TaskRepository:
    def get_pending_by_user(self, user_id: int) -> list[Task]:
        stmt = (
            select(Task)
            .where(Task.user_id == user_id)
            .where(Task.is_completed == False)
            .order_by(Task.due_date)
        )
        return self.db.execute(stmt).scalars().all()

    def get_overdue(self) -> list[Task]:
        stmt = (
            select(Task)
            .where(Task.due_date < date.today())
            .where(Task.is_completed == False)
        )
        return self.db.execute(stmt).scalars().all()
```

4. Facilita Cambios de Tecnología

```
# Si cambias de SQLAlchemy a otro ORM, solo cambias repositories
# Los services NO se modifican

# Repository con MongoDB (hipotético)
class TaskRepositoryMongo:
    def __init__(self, collection):
        self.collection = collection
```

```
def get_by_id(self, task_id: str) -> Task | None:
    doc = self.collection.find_one({"_id": task_id})
    return Task(**doc) if doc else None
```

✗ Cuándo NO Usar Repository Pattern

El patrón agrega complejidad. No siempre es necesario:

Situación	¿Usar Repository?
Aplicación simple (CRUD básico)	✗ Probablemente no
Prototipo o MVP rápido	✗ No
Lógica de negocio compleja	✓ Sí
Múltiples fuentes de datos	✓ Sí
Testing unitario extensivo	✓ Sí
Equipo grande	✓ Sí
Posible cambio de ORM/DB	✓ Sí

📌 Responsabilidades por Capa

Capa	Responsabilidad	Conoce
Router	HTTP, validación de request, responses	Schemas, HTTPException
Service	Lógica de negocio, validaciones, orquestación	Models, Repositories
Repository	CRUD, queries, persistencia	SQLAlchemy, Session

✓ Checklist de Comprensión

Antes de continuar, asegúrate de entender:

- ☐ El Repository Pattern abstrae el acceso a datos
- ☐ Services NO deben tener código SQLAlchemy
- ☐ Repositories manejan SOLO operaciones de datos
- ☐ El patrón facilita el testing con mocks/fakes
- ☐ No siempre es necesario (evaluar complejidad)

🔗 Siguiente

En el próximo archivo aprenderemos a crear un **Repositorio Genérico** que evite duplicación de código entre repositorios específicos.

→ [02-repositorio-generico.md](#)